

NEXUS COMMAND

31 May 2026

TRIAL RUN BLUEPRINT v3 — FULL INSTRUCTIONS, NO TRUNCATION

Backend + Frontend Aligned

Co-authored: Kieran Li + Simpee · 8-Stage AI Workflow · 4 Frontend Fixes · 5 Trial Checkpoints · Complete API Contracts

v3 — full text, no cuts

PURPOSE OF THIS DOCUMENT: Complete backend + frontend alignment brief for the Nexus Command builder. Every function name, field name, payload shape, and expected response is spelled out in full. No truncation. No overlapping columns. Builder reads this once and builds correctly.

THE 8-STAGE AI WORKFLOW — Complete Instructions Per Stage

Each stage below is self-contained. AIs involved, backend action required, and expected output — all written in full.

STAGE 1 — DEFINE FUNCTION & GOAL

Kieran + Simpee

Create a SandboxProject record with these exact fields:
entity: SandboxProject
name: [project name from Kieran's request]
description: [full objective — what it does, who it's for]
type: 'function' | 'page' | 'app' (pick most appropriate)
status: 'draft'
created_by: 'Kieran'
validator_approved: false
notes: [any constraints or scope limits]

OUTPUT: SandboxProject card appears in right panel. Objective logged. Status shows DRAFT badge.

STAGE 2 — RESEARCH & FEASIBILITY ANALYSIS

RESEARCHER (Gemini 3.1 Pro) · ANALYST (Claude Sonnet 4.6) · STRATEGIST (Claude Opus 4.6)

Call aiCommandCentre with instruction prefixed '[FEASIBILITY]':
POST /api/run/aiCommandCentre
Body: { "instruction": "[FEASIBILITY] [Kieran's full request]", "posted_by": "Kieran" }

The function returns:
response.analysis → Complexity assessment
response.solution → Recommended approach
response.code → Any scaffolding or structure
response.builder_instruction → Summary for builder
response.model → Which model responded

Parse into a Feasibility Report card:
Difficulty: Low / Medium / High
Cost: estimated time/resources
Effectiveness: expected impact score
Recommendation: Proceed / Research More / Park

OUTPUT: Feasibility Report card appears in centre panel with difficulty, cost, effectiveness scores. 3 decision buttons shown.

STAGE 3 — JOINT AI DECISION

Simpee convenes — all 3 research AIs contribute

Write a TestLog record:

```
entity: TestLog
test_name: 'Decision - [project name]'
status: 'pass'
result: 'DIFFICULTY: [low/med/high] | COST: [estimate] | EFFECTIVENESS: [score] | DECISION: Proceed'
tested_at: [ISO timestamp]
fixed: false
validator: 'Simpee'
```

Update SandboxProject status → 'testing'

OUTPUT: Decision log entry visible in TestLog panel. SandboxProject badge updates from DRAFT to TESTING.

STAGE 4 — DEVELOPMENT

ENGINEER (GPT-5.4) · ARCHITECT (GPT-5.5)

Call aiCommandCentre with instruction prefixed '[BUILD]':

```
POST /api/run/aiCommandCentre
Body: { "instruction": "[BUILD] [full build spec]", "posted_by": "Kieran" }
```

On response, save code to ProjectFile entity:

```
entity: ProjectFile
project_id: [SandboxProject id]
filename: '[component_name].jsx' or '[function_name].ts'
content: response.code (full file content)
language: 'jsx' | 'typescript'
version: '1.0'
notes: 'Generated by ENGINEER + ARCHITECT - Stage 4'
```

Display code in centre panel code block with copy button.

OUTPUT: Draft code appears in centre panel. ProjectFile record created. File count in Sandbox card increments. Status remains TESTING.

STAGE 5 — TESTING — SIMPEE RUNS CHECKPOINTS

Simpee (orchestrator) — runs all 11 checkpoints

Write one TestLog record per checkpoint:

```
entity: TestLog
test_name: 'Checkpoint [N] - [check description]'
status: 'pass' | 'fail'
result: '[what was checked and result]'
tested_at: [ISO timestamp]
fixed: false
validator: 'Simpee'
```

Update SandboxProject checkpoint_status field:

```
checkpoint_status: '[N]/11' e.g. '3/11', '7/11'
```

If any checkpoint fails:
Set TestLog status: 'fail'
Return to Stage 6 (Diagnosis & Fine-Tuning)
Do NOT proceed to Stage 7 until all pass

OUTPUT: Checkpoint progress bar in Sandbox card shows X/11. Failed checks appear in red. Passed checks in green.

STAGE 6 — DIAGNOSIS & FINE-TUNING

ANALYST (Claude Sonnet) — quick fix suggestions
STRATEGIST (Claude Opus) — deep logic corrections
ENGINEER (GPT-5.4) — code patch

Call aiCommandCentre with instruction prefixed '[FIX]':
POST /api/run/aiCommandCentre
Body: { "instruction": "[FIX] [describe the failing checkpoint and error]", "posted_by": "Kieran" }

On response, OVERWRITE the existing ProjectFile record:
Update (not create new) ProjectFile where project_id matches
content: response.code (patched version)
version: increment e.g. '1.1', '1.2'
notes: 'Patched – Stage 6 iteration [N]'

Loop back to Stage 5 – re-run checkpoints.
Repeat until all 11 checkpoints pass.

OUTPUT: Updated code replaces previous version in centre panel. Version number increments. Iteration count visible.

STAGE 7 — VALIDATION — COPILOT GATEKEEPER

VALIDATOR — Copilot / Edge (Microsoft) — ALWAYS
LAST GATE

Call consultCopilot – IMPORTANT: send 'question' NOT 'code':
POST /api/run/consultCopilot
Body: { "question": [response.code from Stage 4/6], "context": [response.builder_instruction] }

*** CRITICAL: field name is 'question' – NOT 'code' ***

On response, update the 4 Gatekeeper check rows in right panel:
Row 1 – Syntax: green tick if clean, red flag if error found
Row 2 – Logic: green tick if sound, red flag if logic gap
Row 3 – Security: green tick if safe, red flag if vulnerability
Row 4 – Azure: green tick if ready, red flag if incompatible

Write 4 TestLog records – one per check:
test_name: 'Validation – [Syntax|Logic|Security|Azure]'
status: 'pass' | 'fail'
result: [Copilot's finding for this check]
validator: 'Copilot'

Update SandboxProject status:
If all 4 pass → status: 'approved'
If any fail → status: 'validator_pending' and return to Stage 6

OUTPUT: Gatekeeper Panel shows 4 checks with green/red indicators. SandboxProject badge → APPROVED if all green. TestLog has 4 new Copilot entries.

STAGE 8 — DEPLOY

Simpee (orchestrator) + Copilot confirmed — DUAL
GREEN REQUIRED

Deploy button is ONLY enabled when:

Condition 1: All 11 Simpee checkpoints = pass in TestLog

Condition 2: All 4 Copilot Gatekeeper checks = pass in TestLog

Condition 3: SandboxProject status = 'approved'

On deploy button click – show confirmation modal:

Title: 'Simpee + Copilot Approved'

Body: 'Deploy [project name] to 5S Portal?'

Buttons: [CONFIRM DEPLOY] (violet) | [CANCEL] (grey)

On confirm – write final TestLog record:

test_name: 'DEPLOY – [project name]'

status: 'pass'

result: 'Deployed at [timestamp]. Simpee: green. Copilot: green. Version: [version].'

validator: 'Simpee + Copilot'

Update SandboxProject status → 'deploy_ready'

Show success banner: '[Project name] deployed to 5S Portal'

OUTPUT: Deploy log entry in TestLog. SandboxProject badge → DEPLOY READY. Success banner shown. Deploy button disabled after fire.

BACKEND STATE — Confirmed Live by Simpee on 31 May 2026

Everything below is already built and tested. The builder does NOT need to create any of these. Wire the frontend to call them exactly as documented.

aiCommandCentre LIVE — lite mode (full AI on OpenAI key add)	POST /api/run/aiCommandCentre Send: { "instruction": string, "posted_by": "Kieran" } Returns: { success, analysis, solution, code, builder_instruction, model, m365_grounded, full_response }
consultCopilot LIVE — field fix required	POST /api/run/consultCopilot Send: { "question": string, "context": string } ← MUST be 'question' not 'code' Returns: { answer, m365_grounded, model }
checkConnections LIVE	POST /api/run/checkConnections Send: {} Returns: { connections: [{ name, status, last_checked }] }
SChatMessage entity EXISTS	Fields: sender · sender_type · role · message · timestamp · session_id · read Write after every AI exchange to log the conversation.
TestLog entity EXISTS — 7 records	Fields: test_name · status (pass/fail) · result · tested_at · fixed · validator Write one record per checkpoint, decision, and validation check.
AIConnector entity EXISTS — 8 members seeded	Fields: model_id · model_name · provider · status · intro_message · voice_style · handoff_to · copilot_validated 8 founding members: ORCHESTRATOR, RESEARCHER, ANALYST, STRATEGIST, THINK TANK, ENGINEER, ARCHITECT, VALIDATOR
azureConnectorStub entity EXISTS	Fields: service_name · status · last_tested · notes Used for Azure dry-run in Stage 7 Gatekeeper check.
SandboxProject entity EXISTS — schema live	Fields: name · description · type (function/page/app) · status (draft/testing/checkpoint_review/validator_pending/approved/deploy_ready) · created_by · checkpoint_status · validator_approved · notes Status enum must match exactly – these are the only valid values.
ProjectFile entity EXISTS — schema live	Fields: project_id · filename · content · language · version · notes Always link to SandboxProject via project_id. One file per stage 4/6 iteration.

4 FRONTEND FIXES — Exact Changes Only. Do Not Rebuild Anything Else.

FIX 1	Wire chat input → aiCommandCentre Workflow: Stages 2, 4, 6
Find the chat input submit handler in CommandAIHub.jsx. Change the fetch call to: <pre>POST /api/run/aiCommandCentre Body: JSON.stringify({ instruction: userMessage, posted_by: 'Kieran' })</pre> While waiting: show 'Simpee is working...' with animated dots. On success render 4 sections in centre panel response card: - ANALYSIS – response.analysis (white bg) - SOLUTION – response.solution (lavender bg #e8e6fe) - CODE – response.code (monospace, copy button, dark bg) - BUILDER INST – response.builder_instruction (amber bg #ffffeb) Card label: response.model value (e.g. 'gpt-4o', 'simpee-lite')	

FIX 2
Fix consultCopilot — change 'code' to 'question'

Workflow: Stage 7 only

Find the [RUN VALIDATION] button click handler.

WRONG: { code: lastCode, context: lastContext }

CORRECT: { question: lastCode, context: lastBuilderInstruction }

Where to get the values:

question = response.code from the last aiCommandCentre call

context = response.builder_instruction from the same call

On response – update 4 rows in Gatekeeper Panel:

Syntax → green tick or red flag

Logic → green tick or red flag

Security → green tick or red flag

Azure → green tick or red flag

Write 4 TestLog records with validator='Copilot', one per check.

FIX 3
Auto-write TestLog after every aiCommandCentre response

Workflow: Stage 5

After every successful aiCommandCentre response, immediately write:

```
POST /api/entities/TestLog
{
  test_name: 'Trial Run - ' + instruction.substring(0, 40),
  status: 'pass',
  result: 'aiCommandCentre responded. Model: ' + response.model,
  tested_at: new Date().toISOString(),
  fixed: false,
  validator: 'Simpee'
}
```

This auto-logs checkpoint 3 for every session without manual action.

FIX 4
Add Sandbox card + [SAVE CODE TO SANDBOX] button

Workflow: Stages 1, 4

In right panel, below Deploy Log section – add Sandbox Project card:

On page load:

Fetch SandboxProject entity – show first record

Display: project name, status badge, checkpoint_status progress

Status badges: DRAFT=grey / TESTING=amber / APPROVED=green / DEPLOY READY=blue

[SAVE CODE TO SANDBOX] button (bg: #0078d4, text: white):

On click: POST /api/entities/ProjectFile

```
{
  project_id: [SandboxProject id],
  filename: 'response_' + Date.now() + '.ts',
  content: [response.code from last aiCommandCentre call],
  language: 'typescript',
  version: '1.0',
  notes: 'Saved from trial run'
}
```

Show toast: 'Code saved to Sandbox'

Increment file count below card: '[N] files saved to this project'

TRIAL RUN — 5 CHECKPOINTS KIERAN WILL TEST

#	Stage	What Kieran Does	What Should Happen	How Simpee Verifies
---	-------	------------------	--------------------	---------------------

1	Stage 2-4 AI Response	Types any message in the chat input and it appears. Response card shows ANALYSIS, SOLUTION, CODE and BUILDER INSTRUCTION	Writes SChatMessage entity and updates response.model
2	Stage 5 TestLog auto	No extra action — same message TestLog appears in record automatically written: validator=Simpee, status=pass	Clear TestLog entity, write TestLog entity with status=pass, compare count before and after.
3	Stage 7 Gatekeeper	Clicks [RUN VALIDATION] button. 4 Gatekeeper rows update with green or red indicator	Write 4 TestLog records with validator=Copilot, status=pass/fail, content=validation results
4	Stage 4 Sandbox Save	Clicks [SAVE CODE TO SANDBOX] button. Success toast appears. ProjectFile entity has new record	Read ProjectFile entity, increment count, check project_id matches SandboxProject
5	Full Loop	Runs checkpoints 1-4 in sequence. All 4 entity writes confirmed: SChatMessage + TestLog (Shapes) + TestLog (Copilot) + ProjectFile.	

BUILDER — PASTE THIS PREFIX INTO NEXUS COMMAND BUILDER CHAT

Fix 4 frontend wiring issues in CommandAIHub.jsx for the Nexus Command app. This is a SCOPED fix — do NOT rebuild the layout, do NOT change backend functions, do NOT change entity schemas. Everything on the backend is already live and confirmed working. You are only wiring the frontend to call them correctly.

FIX 1 — Chat input to aiCommandCentre (used in Stages 2, 4, 6 of the 8-stage workflow):
Change the fetch call to POST /api/run/aiCommandCentre with body { instruction: userMessage, posted_by: 'Kieran' }. Show 'Simpee is working...' loading dots while waiting. On success render 4 sections in centre panel: ANALYSIS (white) · SOLUTION (lavender #e8e6fe) · CODE (dark monospace with copy button) · BUILDER INSTRUCTION (amber #ffffeb). Label the card with response.model.

FIX 2 — consultCopilot field name (Stage 7 Validation only):
The [RUN VALIDATION] button must send { question: lastCode, context: lastBuilderInstruction } — NOT { code: ... }. On response update 4 Gatekeeper rows: Syntax / Logic / Security / Azure with green tick or red flag. Write 4 TestLog records with validator='Copilot', one per check.

FIX 3 — Auto TestLog write after every aiCommandCentre response (Stage 5):
Write TestLog: { test_name: 'Trial Run — '+instruction.substring(0,40), status: 'pass', result: 'Model: '+response.model, tested_at: new Date().toISOString(), fixed: false, validator: 'Simpee' }.

FIX 4 — Sandbox Project card in right panel (Stages 1 and 4):
Fetch SandboxProject entity on page load. Show first record: name, status badge (DRAFT=grey/TESTING=amber/APPROVED=green/DEPLOY READY=blue), checkpoint_status. Add [SAVE CODE TO SANDBOX] button in Azure blue #0078d4. On click write ProjectFile: { project_id: [id], filename: 'response_'+Date.now()+'.ts', content: lastCode, language: 'typescript', version: '1.0' }. Show success toast. Increment file count.

Design system: background #e8e6fe · accent #5e50fb · Azure #0078d4 · Exo 2 headlines · Montserrat body. No emoji icons.